

LAB # 11

Array Implementation in MIPS Assembly Language

OBJECTIVE

To implement array as an abstract data structure in MIPS assembly language.

Theory

Not like high level languages, Assembly language has no notion of an array at all. Arrays like variables are treated as a block of memory that could be allocated with a single directive, where the first element is given a label.

The array as the most important and most general data structure has the following properties:

1. All elements must be the same size. The array is a homogeneous data structure.
2. The size of an array is fixed. The number of elements is fixed.
3. A label (address) is tied to the first element of the array.
4. Traversing each element of an array needs an index or indices and the label as the array's name.

ARRAY DECLARATION:

With reference to the above properties, in assembly language to declare an array it requires:

1. A label name,
2. The number of elements,
3. The size of each element,
4. The initial value of each element.

EXAMPLE#1:

```
.data
A01: .byte 'a', 'k', 'p', 5      #array of 4 bytes: {'a', 'k', 'p', 5}
A02: .word 5, 6, -9, 7          # array of 4 words: {5, 6, -9, 7}
B02: .space 40                  #allocate 40 consecutive bytes, with storage uninitialized
B03: .word -1:30                # allocate 30 consecutive words with each element
                                # initialized with -1.
```

EXAMPLE#2:

The following code fragment is to access the sixth element of table1:

```
.data
table1: .word 4, 5, 6, 7, 8, 9, 10, 21

.text
main:
    la  $t0, table1    # Load address of table1 into $t0

    lw  $t1, 20($t0)    # Load 6th element (value 9)
```

EXAMPLE#3:

```
.data
array: .word 0:12      # 12 integers, start at 0

.text
main:
    # Update array[4] = -23
    li  $t0, 4          # $t0 = index = 4
    la  $t1, array       # $t1 = base address of array
    mul $t2, $t0, 4      # $t2 = offset = index * 4
    add $t3, $t1, $t2    # $t3 = address of array[4]
    li  $t4, -23         # $t4 = value to store
    sw  $t4, ($t3)       # store -23 at array[4]
```

TASKS:

1. Display the contents of the 5th element of an array of 10 integers
2. Implementation of a code to re-initialize each element of an array of 10 integers to be the value "18"